Práctica 6
Operaciones transaccionales

1) Objetivos

- Ampliar el proyecto Biblioteca añadiendo las tablas usuario y prestamo.
- **Implementar los casos de uso "prestar un libro" y "devolver un libro" como operaciones transaccionales** (todo o nada).
- Coordinar dos repositorios JDBC desde un Service, usando la MISMA Connection para que commit/rollback funcionen.
- **Evitar préstamos activos duplicados del mismo libro** con una restricción/índice en PostgreSQL.
- Probar por consola y con consultas SQL que la base de datos queda consistente.

2) Qué se entrega

Capturas de pantalla de:

- Archivo SQL DDL (guardado en src/main/resources/db/ddl_practica6_prestamo.sql).
- Clases model: Usuario.java y Prestamo.java.
- Repositorio JDBC: JdbcPrestamoRepository.java.
- Cambios en JdbcLibroRepository.java (métodos con Connection compartida).
- Service: PrestamoService.java (transacción real).
- App de prueba: PrestamoDemo.java.
- Verificación en pgAdmin (estado de libro y tabla prestamo).

3) Qué debe verse en consola (orientativo)

Ejemplo (los id pueden variar):

```
=== PRESTAR (OK) ===
Creado: Prestamo{id=..., usuarioId=1, libroId=1, ...}

=== PRESTAR MISMO LIBRO (DEBE FALLAR) ===
Esperado: Error al prestar (rollback realizado): El libro no está disponible (o no
existe). id=1

=== DEVOLVER (OK) ===
Devuelto préstamo id=...
```

4) Preparación BD

En pgAdmin abre Query Tool en tu base de datos (biblioteca) y ejecuta el script DDL de abajo.

Asegúrate de que ya existe la tabla libro (de prácticas anteriores).

4.1 Script DDL

Guárdalo en: src/main/resources/db/ddl_practica6_prestamo.sql

```
-- Tabla usuario
CREATE TABLE IF NOT EXISTS usuario (
  id BIGSERIAL PRIMARY KEY,
  nombre VARCHAR(120) NOT NULL,
  email VARCHAR(120) UNIQUE
);

-- Tabla prestamo
CREATE TABLE IF NOT EXISTS prestamo (
  id BIGSERIAL PRIMARY KEY,
  usuario_id BIGINT NOT NULL REFERENCES usuario(id),
  libro_id BIGINT NOT NULL REFERENCES libro(id),
  fecha_inicio DATE NOT NULL,
  fecha_fin DATE NOT NULL,
  fecha_devolucion DATE,
  CONSTRAINT ck_fechas CHECK (fecha_fin >= fecha_inicio)
);

-- Evitar dos préstamos ACTIVOS del mismo libro
CREATE UNIQUE INDEX IF NOT EXISTS uq_prestamo_libro_activo
ON prestamo(libro_id)
WHERE fecha_devolucion IS NULL;

-- Datos de prueba: usuarios (alumnado)
INSERT INTO usuario(nombre,email) VALUES
('Oumnia', 'oumnia@correo.local'),
('Maria Fernanda', 'maria.fernanda@correo.local'),
('Saad', 'saad@correo.local'),
('Adrian Fernandez', 'adrian.fernandez@correo.local'),
('David Gallardo', 'david.gallardo@correo.local'),
('Silvia', 'silvia@correo.local'),
('Victor', 'victor@correo.local'),
('Javier Faustino', 'javier.faustino@correo.local'),
('Javier Herranz', 'javier.herranz@correo.local'),
('Daniel Jimenez', 'daniel.jimenez@correo.local'),
('Raul', 'raul@correo.local'),
('Juanma', 'juanma@correo.local'),
('Mario Losada', 'mario.losada@correo.local'),
('Javier Machuca', 'javier.machuca@correo.local'),
('Javier Montilla', 'javier.montilla@correo.local'),
('Jorge Moreno', 'jorge.moreno@correo.local'),
('Alvaro Moreno', 'alvaro.moreno@correo.local'),
('Salma', 'salma@correo.local'),
('Mateo', 'mateo@correo.local'),
('Daniel Muñoz', 'daniel.munoz@correo.local'),
('Victor Pavon', 'victor.pavon@correo.local'),
```

```
('Jhonny', 'jhonatan@correo.local'),
('Victor Reina', 'victor.reina@correo.local'),
('Mario Rodriguez', 'mario.rodriguez@correo.local'),
('Alvaro Sanchez', 'alvaro.sanchez@correo.local'),
('Juan Torres', 'juan.torres@correo.local'),
('Adrian Torres', 'adrian.torres@correo.local')
ON CONFLICT DO NOTHING;

-- Datos de prueba: libros (fantasía y ciencia ficción)
-- Nota: se asume tabla libro(titulo, isbn, anio, disponible). Ajusta si tu tabla
difiere.
INSERT INTO libro(titulo, isbn, anio, disponible) VALUES
('Dune', 'SCI-0001', 1965, true),
('Fundación', 'SCI-0002', 1951, true),
('Neuromante', 'SCI-0003', 1984, true),
('Hyperion', 'SCI-0004', 1989, true),
('El juego de Ender', 'SCI-0005', 1985, true),
('La mano izquierda de la oscuridad', 'SCI-0006', 1969, true),
('Snow Crash', 'SCI-0007', 1992, true),
('Ready Player One', 'SCI-0008', 2011, true),
('El problema de los tres cuerpos', 'SCI-0009', 2006, true),
('Blade Runner (¿Sueñan los androides...? )', 'SCI-0010', 1968, true),
('El nombre del viento', 'FAN-0001', 2007, true),
('El temor de un hombre sabio', 'FAN-0002', 2011, true),
('Mistborn: El imperio final', 'FAN-0003', 2006, true),
('El pozo de la ascensión', 'FAN-0004', 2007, true),
('El héroe de las eras', 'FAN-0005', 2008, true),
('Elantris', 'FAN-0006', 2005, true),
('La comunidad del anillo', 'FAN-0007', 1954, true),
('Las dos torres', 'FAN-0008', 1954, true),
('El retorno del rey', 'FAN-0009', 1955, true),
('Juego de tronos', 'FAN-0010', 1996, true),
('Choque de reyes', 'FAN-0011', 1998, true),
('Tormenta de espadas', 'FAN-0012', 2000, true),
('La rueda del tiempo: El ojo del mundo', 'FAN-0014', 1990, true),
('Harry Potter y la cámara secreta', 'FAN-0016', 1998, true),
('La princesa prometida', 'FAN-0017', 1973, true),
('El archivo de las tormentas: El camino de los reyes', 'FAN-0018', 2010, true),
('La historia interminable', 'FAN-0019', 1979, true),
('La quinta estación', 'SCI-0011', 2015, true)
ON CONFLICT DO NOTHING;
```

5) Código

En este punto verás código con la palabra RELLENAR.

Tu tarea es sustituir cada RELLENAR por el código correcto sin cambiar el resto.

Regla de oro de esta práctica:

- El Service abre UNA Connection y se la pasa a los repos.
- Los repos NO deben abrir conexiones nuevas si forman parte de la transacción.

5.1 Usuario.java

Ruta: src/main/java/com/dam/ad/model/Usuario.java

Esta clase es un POJO: solo guarda datos.

```
package com.dam.ad.model;

public class Usuario {
    private Long id;
    private String nombre;
    private String email;

    public Usuario(String nombre, String email) {
        this.nombre = RELLENAR;
        this.email  = RELLENAR;
    }

    public Usuario(Long id, String nombre, String email) {
        this.id     = id;
        this.nombre = nombre;
        this.email  = email;
    }

    public Long getId() { return id; }
    public String getNombre() { return RELLENAR; }
    public String getEmail()  { return RELLENAR; }
    public void setId(Long id) { this.id = id; }
}
```

5.2 Prestamo.java

Ruta: src/main/java/com/dam/ad/model/Prestamo.java

Nota: en BD libro_id es BIGINT, pero en tu proyecto Libro usa int. Para no romper nada, usamos int en Java para libroId.

```
package com.dam.ad.model;

import java.time.LocalDate;

public class Prestamo {
    private Long id;
    private Long usuarioId;
    private Integer libroId;
    private LocalDate fechaInicio;
    private LocalDate fechaFin;
    private LocalDate fechaDevolucion;

    public Prestamo(Long usuarioId, Integer libroId, LocalDate fechaInicio, LocalDate
fechaFin) {
        this.usuarioId = RELLENAR;
        this.libroId = RELLENAR;
        this.fechaInicio = RELLENAR;
        this.fechaFin = RELLENAR;
    }

    public Long getId() { return id; }
    public Long getUsuarioId() { return usuarioId; }
    public Integer getLibroId() { return libroId; }
    public LocalDate getFechaInicio() { return fechaInicio; }
    public LocalDate getFechaFin() { return fechaFin; }
    public LocalDate getFechaDevolucion() { return fechaDevolucion; }

    public void RELLENAR { this.id = id; }
    public void RELLENAR (LocalDate fechaDevolucion) { this.fechaDevolucion =
fechaDevolucion; }
}
```

5.3 JdbcPrestamoRepository.java

Ruta: src/main/java/com/dam/ad/repository/jdbc/JdbcPrestamoRepository.java

Recuerda: el repositorio recibe Connection desde el Service (no abre Db.getConnection()).

```
package com.dam.ad.repository.jdbc;

import com.dam.ad.model.Prestamo;
import java.sql.Connection;
import java.sql.SQLException;

public class JdbcPrestamoRepository {

    public Prestamo insert(Connection con, Prestamo p) throws SQLException {
        String sql = """
                INSERT INTO prestamo(usuario_id, libro_id, fecha_inicio, fecha_fin)
                VALUES (?, ?, ?, ?)
                RETURNING id
                """;

        try (var ps = con.prepareStatement(sql)) {
            ps.setLong(RELLENAR, p.getUsuarioId());
            ps.setInt(RELLENAR,  p.getLibroId());
            ps.setObject(RELLENAR, p.getFechaInicio());
            ps.setObject(RELLENAR, p.getFechaFin());

            try (var rs = ps.RELLENAR()) {
                RELLENAR
                p.setId(rs.getLong(RELLENAR));
                return p;
            }
        }
    }

    public long marcarDevuelto(Connection con, int libroId) throws SQLException {
        String sql = """
                UPDATE prestamo
                SET fecha_devolucion = CURRENT_DATE
                WHERE libro_id = ? AND RELLENAR
                RETURNING id
                """;

        try (var ps = con.prepareStatement(sql)) {
            ps.setInt(RELLENAR, libroId);
            try (var rs = ps.executeQuery()) {
                if (!rs.next()) throw new IllegalStateException("No hay préstamo
activo para libro id=" + libroId);
                return rs.getLong("id");
            }
        }
    }
}
```

5.4 Métodos necesarios en JdbcLibroRepository

Ruta: src/main/java/com/dam/ad/repository/jdbc/JdbcLibroRepository.java

```
// imports necesarios
import java.sql.Connection;
import java.sql.SQLException;

// ¿Está disponible el libro?
public boolean isDisponible(Connection con, int libroId) throws SQLException {
    String sql = "SELECT RELLENAR FROM libro WHERE id = ?";
    try (var ps = con.prepareStatement(sql)) {
        ps.setInt(RELLENAR, libroId);
        try (var rs = ps.executeQuery()) {
            if (!rs.next()) return false;
            return rs.getBoolean(RELLENAR);
        }
    }
}

// Actualizar disponibilidad
public boolean updateDisponible(RELLENAR) throws SQLException {
    String sql = "UPDATE libro SET disponible = ? WHERE id = ?";
    try (var ps = con.prepareStatement(sql)) {
        ps.setBoolean(RELLENAR, disponible);
        ps.setInt(RELLENAR, libroId);
        return ps.executeUpdate() == RELLENAR;
    }
}
```

5.5 PrestamoService.java (la transacción real)

Ruta: src/main/java/com/dam/ad/service/PrestamoService.java

```
package com.dam.ad.service;

import com.dam.ad.db.Db;
import com.dam.ad.model.Prestamo;
import com.dam.ad.repository.jdbc.JdbcLibroRepository;
import com.dam.ad.repository.jdbc.JdbcPrestamoRepository;

import java.sql.Connection;
import java.time.LocalDate;

public class PrestamoService {

    private final JdbcLibroRepository libroRepo;
    private final JdbcPrestamoRepository prestamoRepo;

    public PrestamoService(JdbcLibroRepository libroRepo, JdbcPrestamoRepository
prestamoRepo) {
        this.libroRepo = libroRepo;
        this.prestamoRepo = prestamoRepo;
    }
```

```
    public Prestamo prestarLibro(long usuarioId, int libroId, LocalDate inicio,
LocalDate fin) {
        try (Connection con = Db.getConnection()) {
            con.setAutoCommit(RELLENAR);
            try {
                if (RELLENAR) {
                    throw new IllegalStateException("El libro no está disponible (o no
existe). id=" + libroId);
                }

                Prestamo p = new Prestamo(usuarioId, libroId, inicio, fin);
                Prestamo guardado = prestamoRepo.insert(con, RELLENAR);

                boolean ok = libroRepo.updateDisponible(con, libroId, RELLENAR);
                if (!ok) throw new IllegalStateException("No se pudo marcar como no
disponible id=" + libroId);

                con.RELLENAR();
                return guardado;

            } catch (Exception e) {
                con.RELLENAR();
                throw e;
            } finally {
                con.setAutoCommit(true);
            }
        } catch (Exception e) {
            throw new RuntimeException("Error al prestar (rollback realizado): " +
e.getMessage(), e);
        }
    }

    public long devolverLibro(int libroId) {
        try (Connection con = Db.getConnection()) {
            con.setAutoCommit(false);
            try {
                long prestamoId = prestamoRepo.RELLENAR(con, libroId);

                boolean ok = libroRepo.updateDisponible(con, libroId, RELLENAR);
                if (!ok) throw new IllegalStateException("No se pudo marcar disponible
id=" + libroId);

                con.commit();
                return prestamoId;

            } catch (Exception e) {
                con.rollback();
                throw e;
            } finally {
                con.setAutoCommit(true);
            }
        } catch (Exception e) {
            throw new RuntimeException("Error al devolver (rollback realizado): " +
e.getMessage(), e);
        }
    }
}
```

5.6 PrestamoDemo.java

Ruta: src/main/java/com/dam/ad/app/PrestamoDemo.java

```
package com.dam.ad.app;

import com.dam.ad.model.Prestamo;
import com.dam.ad.repository.jdbc.JdbcLibroRepository;
import com.dam.ad.repository.jdbc.JdbcPrestamoRepository;
import com.dam.ad.service.PrestamoService;

import java.time.LocalDate;

public class PrestamoDemo {
    public static void main(String[] args) {
        var libroRepo = new JdbcLibroRepository();
        var prestamoRepo = new JdbcPrestamoRepository();
        var service = new PrestamoService(RELLENAR);

        System.out.println("=== PRESTAR (OK) ===");
        Prestamo p = service.RELLENAR(1L, 1, LocalDate.now(),
LocalDate.now().plusDays(14));
        System.out.println("Creado: " + p);

        System.out.println("\n=== PRESTAR MISMO LIBRO (DEBE FALLAR) ===");
        try {
            service.RELLENAR(2L, 1, LocalDate.now(), LocalDate.now().plusDays(7));
        } catch (Exception e) {
            System.out.println("Esperado: " + e.getMessage());
        }

        System.out.println("\n=== DEVOLVER (OK) ===");
        long id = service.RELLENAR(1);
        System.out.println("Devuelto préstamo id=" + id);
    }
}
```

6) Verificación en BD (consultas)

Ejecuta estas consultas en pgAdmin para demostrar que la BD queda consistente:

```
-- Estado del libro (cambia disponible con prestar/devolver)
SELECT id, titulo, disponible FROM libro WHERE id = 1;

-- Préstamos del libro (ver fecha_devolucion al devolver)
SELECT id, usuario_id, libro_id, fecha_inicio, fecha_fin, fecha_devolucion
FROM prestamo
WHERE libro_id = 1
ORDER BY id DESC;
```

7) Problemas típicos

- “Se presta dos veces” → revisa que existe el índice parcial uq_prestamo_libro_activo (y que se ejecutó en la BD correcta).
- “Falla y aun así cambia libro” → estás usando conexiones distintas (algún repo abre su propia conexión).
- “No hay préstamo activo al devolver” → intentas devolver un libro que no está prestado o ya se devolvió.
- “LocalDate da error” → alternativa: ps.setDate(i, java.sql.Date.valueOf(localDate)).
- “Tipos long/int” → usuario_id es long; libroId en vuestro proyecto es int (según prácticas anteriores).

8) Git: commits sugeridos

- Añadir tablas usuario y prestamo + índice único de préstamo activo
- Añadir modelos Usuario y Prestamo
- Añadir JdbcPrestamoRepository (usa Connection compartida)
- Añadir métodos de disponibilidad en libro (usa Connection compartida)
- Añadir PrestamoService con operaciones transaccionales
- Añadir PrestamoDemo para pruebas por consola